

[CS 303] Final Project Spec

Last Updated: Apr 25, 2025

TL;DR - Create a group payment web (or mobile) app such that all members of the group need to accept payment terms in **realtime** for the payment to submit.

	Due Date (all @ 11:59p)	Deliverables to be pushed to your repo on https://classroom.github.com/a/L1CTotQ-
Milestone #1	May 7, 2025	Figma mocks + Design Doc
Milestone #2	Jun 1, 2025 (Hard cutoff – no code can be pushed to repo after 11:59p)	Prototype + Final Presentation
Optional team evals	Jun 11, 2025	Optional team evals (link to be posted)

Problem statement

Ever have to split the bill when eating out with friends but one person just ends up paying while the rest Zelle/Venmo that person back? This group payment app (you can name this product whatever you'd like) would be able to make this process easier.

Concept

In the aforementioned scenario, one person would be responsible for creating a group (using your app) with the bill amount and sending others a link (or using a QR code). This link/QR code would navigate the other group members to a page in the app that shows the amount the group member is responsible for (ex: at the bare minimum the bill would be split evenly across all members of the group). All members of the group must accept the terms (in **realtime**) for the group leader to be able to pay the entire bill using a one time use card (think [NFC](#) like Apple/Google Pay).

Key aspects to focus on

Architecture & design patterns

This doesn't have to be anything overly complex, and certainly doesn't have to be the most optimal. The goal here is to get you thinking about what this system will need in order to make things **realtime**.

Usability

This is meant to be used on mobile devices so think about tap targets/font sizes and efficiency.

Question(s) you might have:

- What actually needs to be implemented?
 - You'll need to implement a prototype for a user to be able to create a group, split the bill, get realtime group member acceptance, and generate a fake "credit card" that can be used to pay the bill (the actual payment part is not necessary).
- Aren't NFCs for web apps on mobile devices currently ultra experimental?
 - Yes, very, but this isn't the main focal point of the project. I want you to think through the various aspects that would go into this system instead. The goal is to get you more familiar with breaking a complex problem into smaller pieces and then thinking about how each piece should come together (architecture) and be designed for robustness/efficiency.
 - But, If you'd like to try out the Web NFC API, go for it!
 - Here's a [doc for NFC for Chrome on Android](#)
 - [Stretch] You can also create a mobile app too using frameworks like React Native/Flutter (see the "Going above and beyond" section below)
- How will we get the user's actual banking information for payment?
 - This feature is beyond the scope of the project but it's something worth looking into and mentioning how you'd approach solving this in your design doc.

Bare Minimum

1. FE using React/Next.js
2. BE with Django or FastAPI

- a. [Stretch] Spring boot

Going above and beyond

1. FE using React Native or Flutter
2. **BE with Firebase**
 - a. **If using Firebase, you'll be expected to have additional above & beyond requirements to ensure technical complexity**
3. Integration with NFC
4. [Stretch] Scannable QR code for group session

Milestone #1: Figma mocks + Draft Design Doc

For this milestone, focus on what a prototype would look like for your app and create a design doc breaking down each major feature..

You'll start with a **design doc** that covers the following:

- How would a user interact with your app? Are there different types of users?
Provide use-case diagram(s)
(<https://www.uml-diagrams.org/use-case-diagrams.html>)
- What is your proposed tech stack? Provide your reasoning for each tool and which others you looked into.
 - Some other frameworks to explore in addition to the ones we discussed in class:
 - FE: Flutter, React Native, Svelte, Solid
 - BE: Ruby on Rails, FastAPI, Express.js
- Initial architecture/design thoughts
 - What will you use to implement the **realtime** aspect of the project?
 - Some examples to look into:
 - GraphQL Subscriptions
 - Ex: Ariadne (for FastAPI or Django)
 - Web Sockets
 - Note: you might see long polling as an approach – do not use this for your project.
- What are the pieces of the app that you'll need to implement? Ex: building a form, tracking user agreements, bill splitting, etc.
- How will you keep track of data?
 - Explore the high level differences between SQL (relational) databases and the various types of NoSQL (non-relational) databases such as document dbs.

You'll also need to provide **Figma mocks** for each of the screens you plan to have with the different app states. Include these mocks as images in your proposal with an explanation of what the mocks represent.

Self-Evaluation Criteria: For Milestone #1, the evaluation criteria is similar to the projects. This is meant to help get your team planning kicked off so that you all can start breaking down the problem statement into manageable pieces. It's especially important that you've considered different tech tooling and how they fulfill the technical requirements.

- You should also consider team bonding activities as needed.

Submitting the milestone: Add a link to the proposal to your **team's repo README (one proposal per team)**. Don't forget to check that the teaching team has comment permissions to your doc.

Milestone #2: Design Doc Refinement & Basic Prototype

This milestone is about building an initial prototype and refining your design doc as needed. This process is meant to be iterative.

Question(s) you might have

Not sure where to start when it comes to building the prototype?

- Start off small and think about delegating work. There are many ways to collaborate. You might [choose to swarm/mob](#) on the
 1. FE and get a basic prototype started with Next.js
 2. BE to serve the FE
 3. Add some initial services to the BE with a database
 4. Then wire up all of BE + FE pieces
- Or you can do something completely different just make sure you each are contributing to the codebase

Evaluation

Prototype - just needs to be barebones (don't worry about making the FE look polished, focus on the actual functionality + architecture)

- Code quality - no lingering log statements, unused code, follows style guides
 - Might be helpful to use linters/[pre-commit hooks](#) to automate this...

- Commits
 - Each team member should contribute meaningful code.
 - If you're [pairing/mobbing](#), then create a separate branch and switch who codes in intervals. After each interval, the person should commit their code to the branch. Once the mob session is done, raise a PR and merge to dev/main (depends on your branching strategy).

Final Presentation

[Sign-up sheet](#)

Note: We may use the entire class time (6p - 7:20p)

Your final presentation will be roughly 6-8 minutes where you'll get a chance to

1. demo your prototype and
2. present how you built your prototype
 1. Think of this as a high-level overview of your design doc (ex: include what tech stack was used and why)
 2. Include technical challenges

Note: No need to go over the problem statement – we will all be working on the same project. Focus on technical complexity.

- I will ask questions about your code and design choices. Be prepared to discuss learning outcomes.
- All team members are expected to present/answer questions (with the exception of those who are away at internships)

For those who have never given a technical presentation before, check out [this post](#)